Design For Test (DFT)

Dr. Paul D. Franzon

Outline

1. *Motivation*
2. *Scan-based testing*
3. *Exhaustive testing, Memory testing, and BIST*
4. *DFT in Synopsys Environment*

References

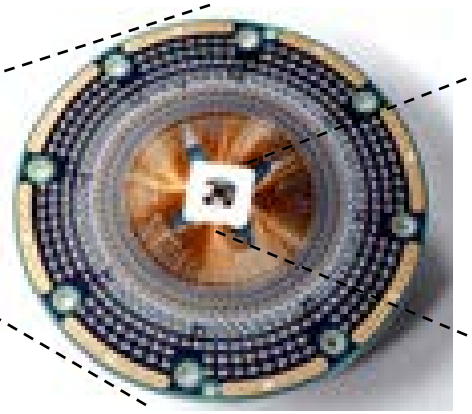
1. *Kurup, Chapter 6.*
2. *M. Smith, Chapter 14.*

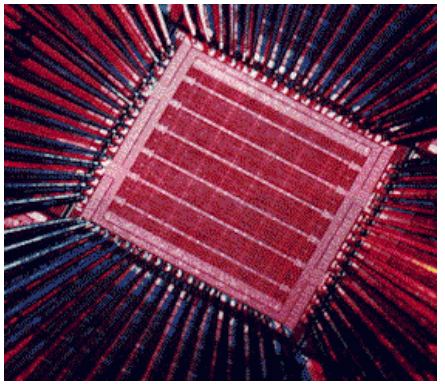
© 2013 Dr. Paul D. Franzon, www.ece.ncsu.edu/erl/faculty/paulf.html

Introduction

Chip test needed after manufacturing so as to determine which parts are free of manufacturing defects

- Main goal is to discover manufacturing defects, not design bugs
- Before wafer dicing:





Chip test

Goal:

- Have a very high probability of finding manufacturing defects
  - Caused by: dust particles, fab control problems
  - Examples: Opens, shorts, transistors/wires out of specs
- While minimizing the cost of finding the defects

Note:

- Features introduced into a chip for DFT can be useful in debugging residual bugs in first round of fabricated parts
- Referred to as "validation"
- Sometimes additional features are added to the chip to help in validation that are not used for DFT
  - Example: Timing and noise measurement circuits

Chip Test and Cost

Cost of test adds up to 40% to the cost of building the chip

- Additional chip area of up to 10% (usually 5%)
- Time spent on the test units
- Typical test steps:
  1. Wafer screening
  2. Die test in wafer form
  3. Die test after singulation and packaging
  4. Interface testing after assembly

However, the cost of releasing a defective chip is very high

- Packaging; Distribution; managing part return;
- Reduction in customer satisfaction
- GOAL: Discover most defects at the factory

Chip Test Methods

Dominant Method:

- Scan Testing using stuck-at fault model

Secondary Methods:

- Functional test
  - Run some of intended functions on chip
  - Given you can't run every function, this does not detect every defect
    - Might only detect 60% of defects (typically 80%)
    - But sometimes used to reduce amount of scan testing requried
- Built In Self Test (BIST)
  - Features added to chip to enable it to test itself
- Delay Fault Testing
  - Testing to ensure chip can run at required speed

Fault Models

Need a fault model so that a simulator can be used to determine the impact of a potential fault

- i.e. Use the simulator to determine how to detect that fault

Actual faults are mainly shorts and opens

- Difficult and messy to model and simulate

In practice using the stuck-at fault model captures the vast majority of "real" faults

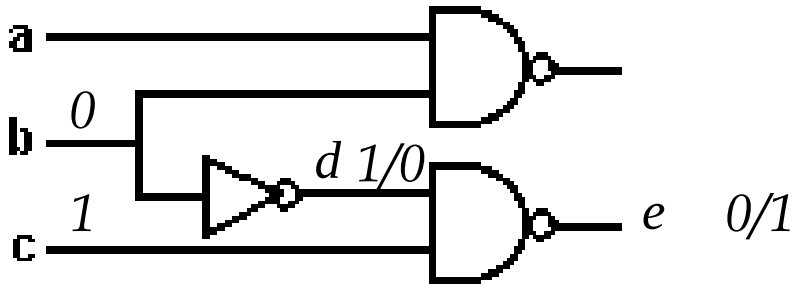
- i.e. Fault = Stuck-At-0 or Stuck-At-1

Using Stuck-At faults

Objectives:

- *To sensitize and propagate faults to scanned outputs.*
- *To achieve high fault coverage (% of potential faults that are detectable)*
- *with fewest possible input choices*

For example,



No-fault/fault

What are the test vectors that will sensitize a S-A-0 at d?

i.e. Drive d =1 : b=0

What test vector will propagate S-A-0 at d to e?

i.e. drive d=1, and cause error at e if d S-A-0 {a,b,c} = {x,0,1}

Note: c=0 would NOT propagate fault to e

D-algorithm

Previous page illustrates the D-algorithm to sensitize and propagate faults

With its extensions:

1. List faults (SA-0/1 for each node)
2. Determine vectors that can be applied to the input of each combinational logic block that sensitize the fault and propagate that detection to an output of the combinational logic block
3. Compress the list of vectors by eliminating overlap
4. Consider eliminating vectors with low fault coverage to reduce test time

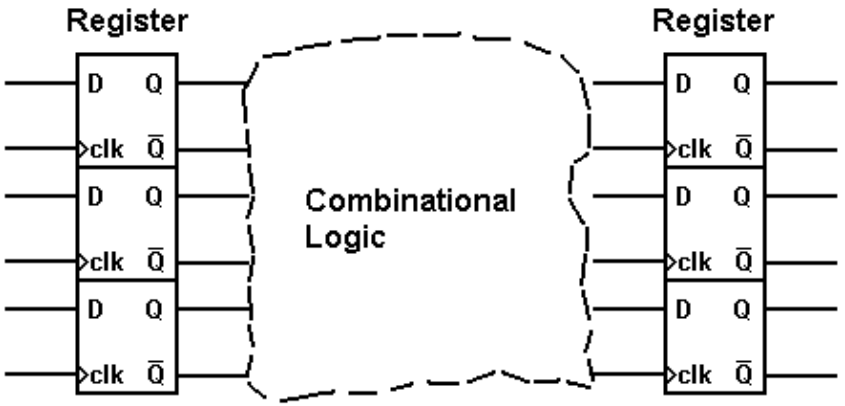
Determines reasonably minimal test vector set that can detect a very high percentage of SA-0 and SA-1 faults at the outputs of the circuit

% of faults detected = fault coverage

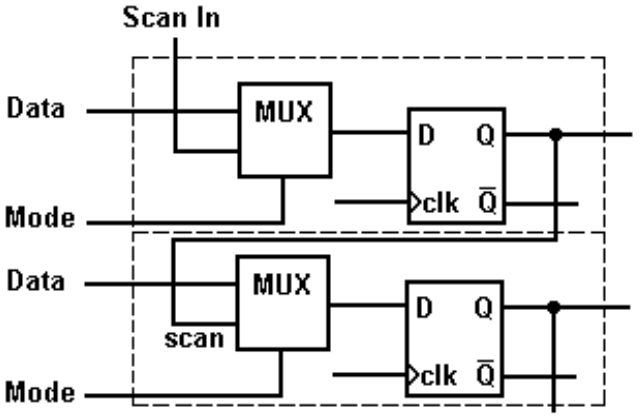
Q: How do we apply the vectors to the combinational logic blocks and observe the outputs from OUTSIDE the chip?

Scan-Based Testing

Synchronous Logic Under Normal Mode of Operation:

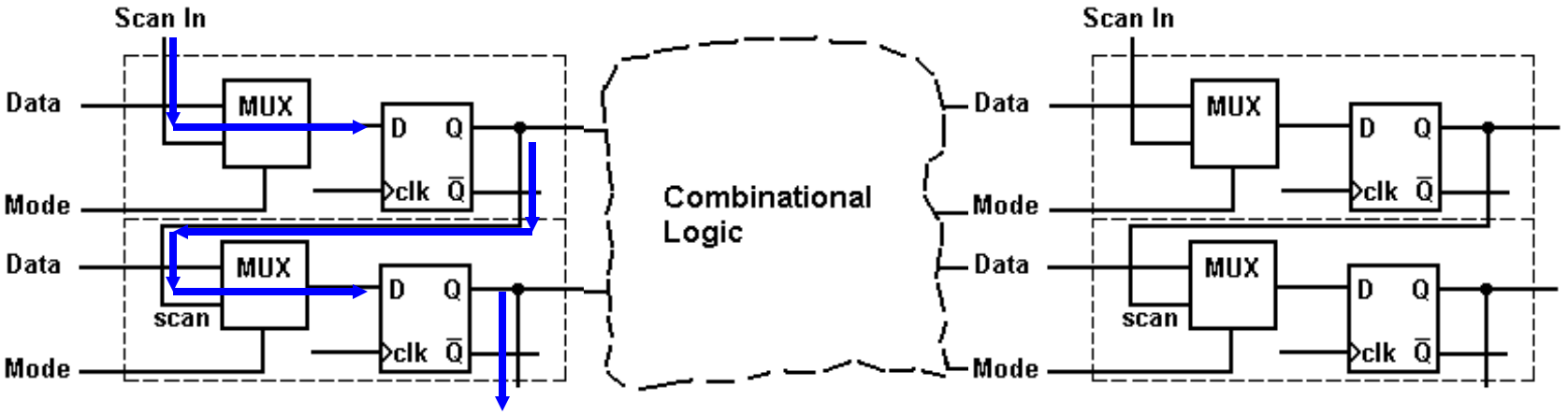


Replace Register Cells with Scan Cell:



Scan-Based Testing...

Test Mode of Operation

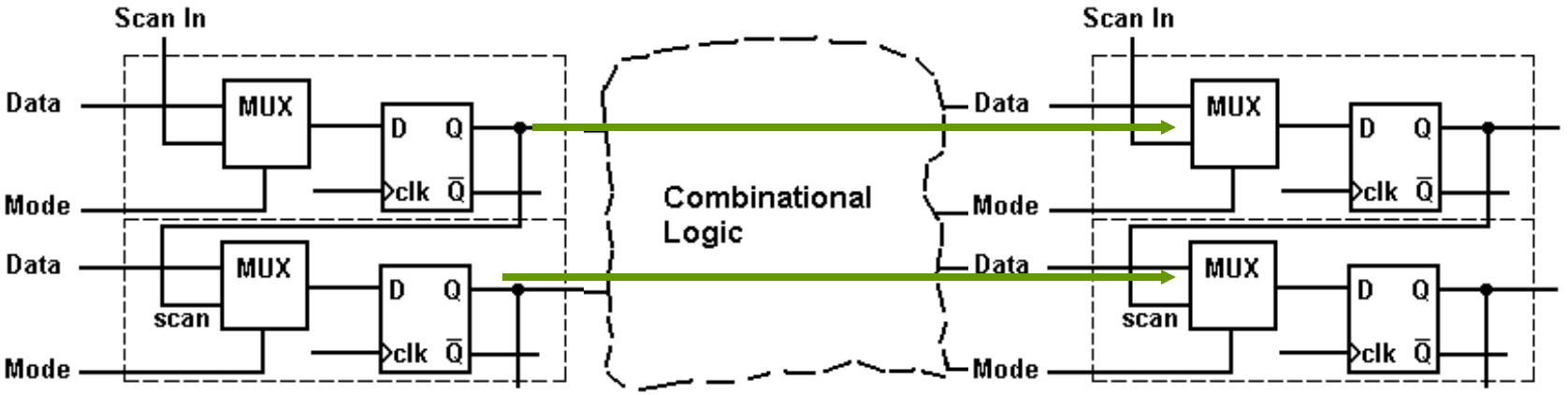


Step 1 : Scan in test vector

Usually applied at the clock rate of the test equipment
Typically much slower than the on-chip clock

Scan-Based Testing...

Test Mode of Operation



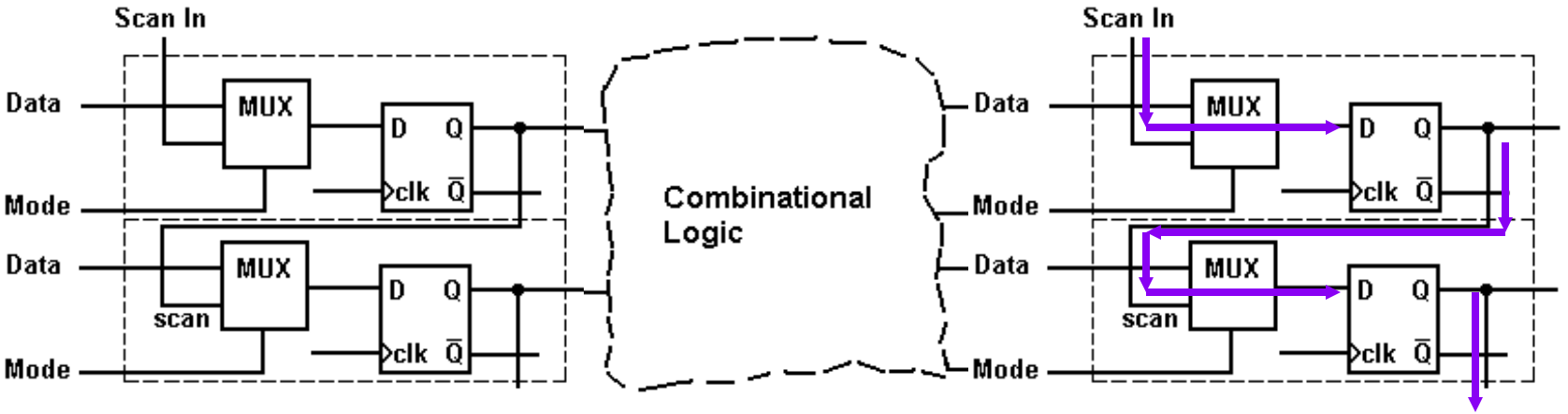
Step 2 : Operate for one clock cycle

Optionally: Use one cycle of the on-chip clock for this step

➔ *Detecting delay faults*

Scan-Based Testing...

Test Mode of Operation



Step 3 : Scan out result vector

Operates at tester clock rate

Permits testing of all combinational logic in a design.

- Flip-flops tested through the scan chain itself

Test Quality

Test Escape

It can be shown that the percentage of parts that are still faulty but are not detected as being faulty, is given as[1]*:*

$$TE = 1 - Y^{(1 - T)}$$

where TE is the test escape, T is fault coverage, Y is the yield (% parts good).

For example, for Y = 50% and T = 95%, what is the test escape? $1-0.5^{0.05}=3\%$

Typically, using the verification vectors (from your Verilog test fixture) for test gives a 80% coverage. What is the test escape then? $1-0.5^{0.2}=13\%$

Typically, fault coverage above 99% give acceptably low test escapes.

[1] *Source: Williams and Brown ("Defect level as a function of fault coverage," IEEE Trans Comp., C-30(12), December 1981, pp. 987-988.*

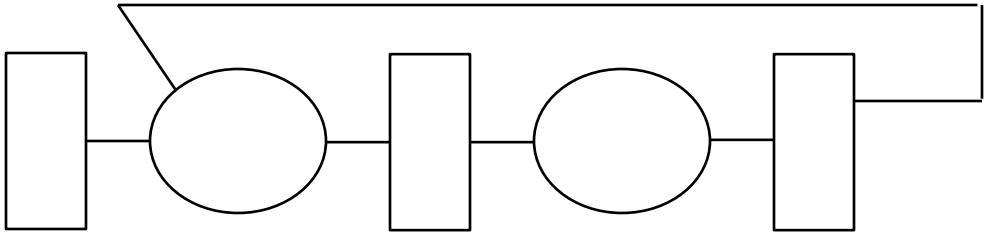
Partial Scan

Full Scan:

- Every flip-flop is included in a scan chain (generally several scan chains on the chip)

Partial Scan:

- Some flip-flops left out
- Works best on synchronous circuits without feedback
  - i.e. Excluded flip-flops treated as logic pass-throughs with a single cycle delay



Sub-module 1 : Summary

Main points

- Every chip has features added to permit each chip to be tested before leaving factory
  - Testing for presence of defects or manufacturing faults
- Main fault model used = stuck-at 0/1
- For each combinational logic block tools determine vectors that sensitize these faults and propagates error to logic output
- Flip-flops configured as scan chains to write in vectors and read out results

© 2013 Dr. Paul D. Franzon, www.ece.ncsu.edu/erl/faculty/paulf.html

Memory Testing

- Memories are exhaustively tested.

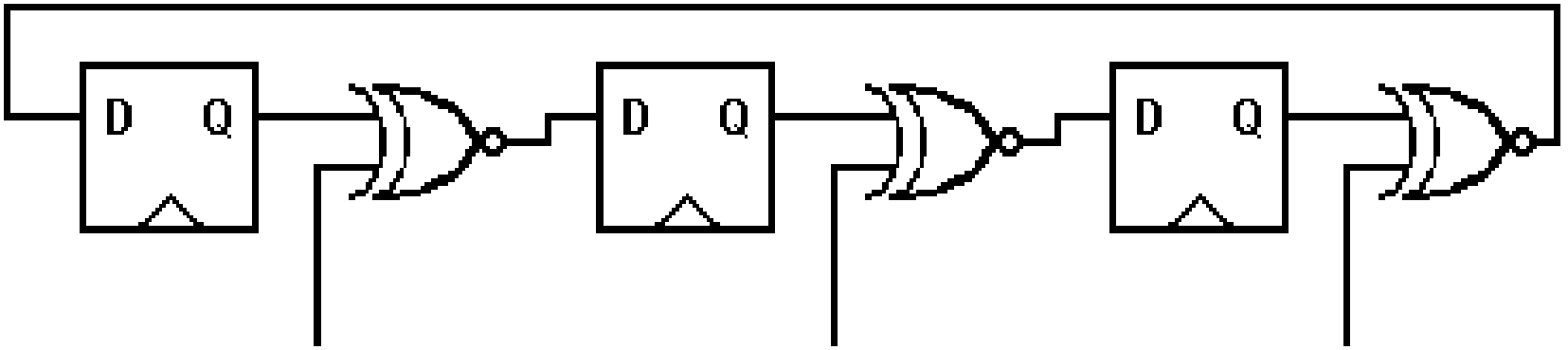
- Memories are tested for opens, shorts at individual locations and shorts between neighboring locations by **marching** test patterns through them.

- Memories are isolated by scan registers for testing purposes.

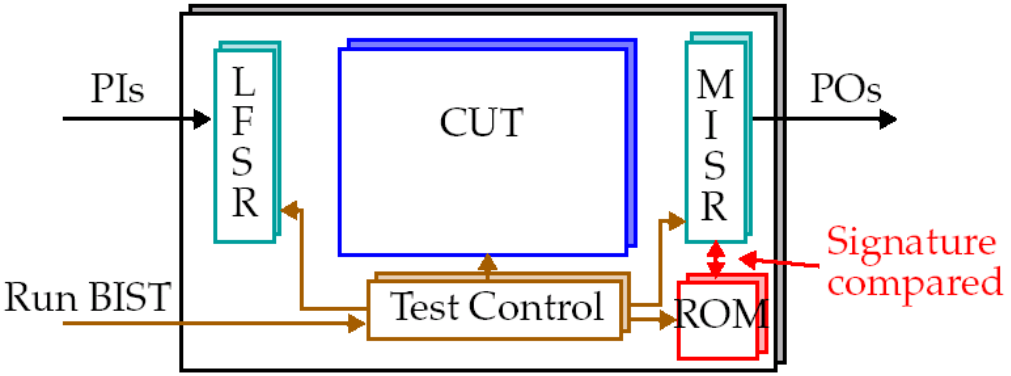
- Example:
  - Step 1: Write 1's to every location, then read
  - Step 2: Write 0's to every location, then read
  - Step 3: Write 1010101010 to every other row in the memory, and 01010101 to intermediate rows and then read
  - Step 4: Write 01010101 and 10101010 and then read

Built-In Self Test (BIST)

- In the most common BIST approach, the scan cells are modified to generate pseudo-random test vectors at the input to a logic block, and then to collect a signature at the output (using a linear feedback shift register).
- BIST takes more silicon area and more cycles (as pseudorandom) but saves on expensive testing units (with lots of memory)
- Also takes less elapsed time as can often be run at full clock rate
- Automatic CAD support available



BIST



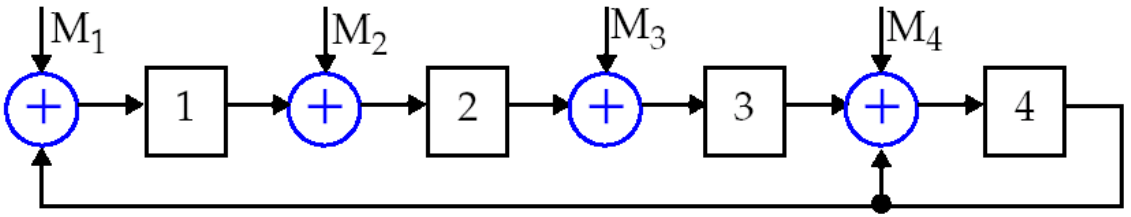
CUT=Circuit Under Test

LFSR:

- Linear Feedback Shift Register
- Used to generate pseudo-random sequence

MISR

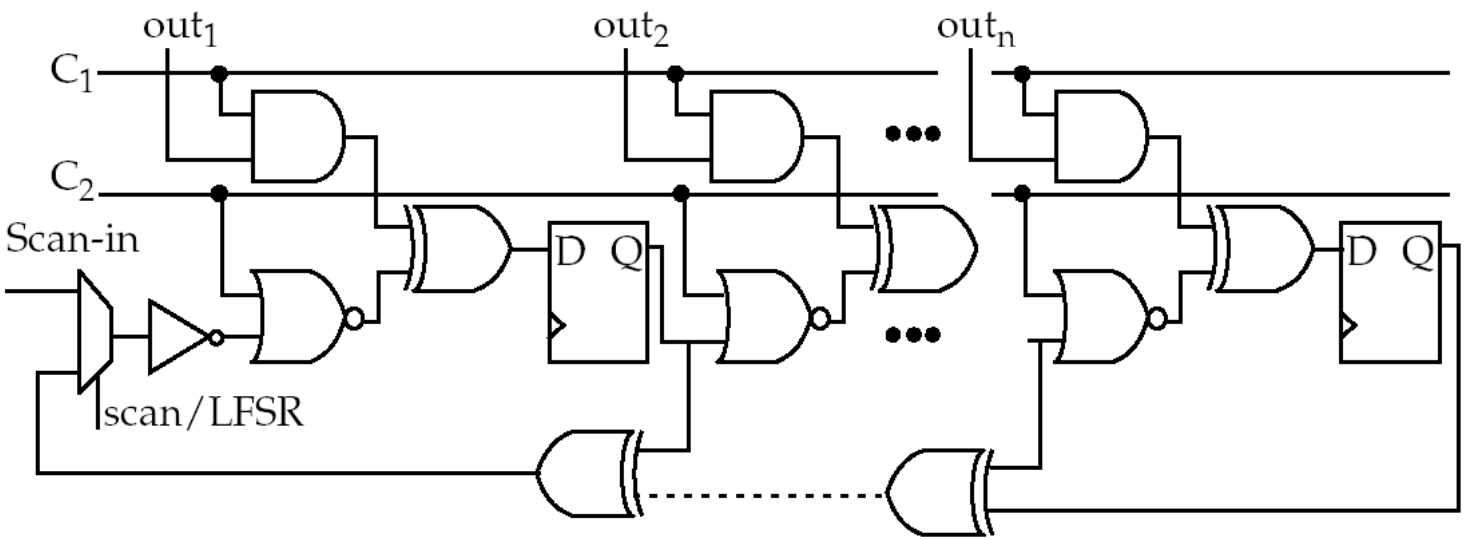
- Multiple Input Signature Register
- Used to generate signature of tested circuit



BILBO

Built-In Logic Block Observer:

- Incorporates BIST into a scan architecture

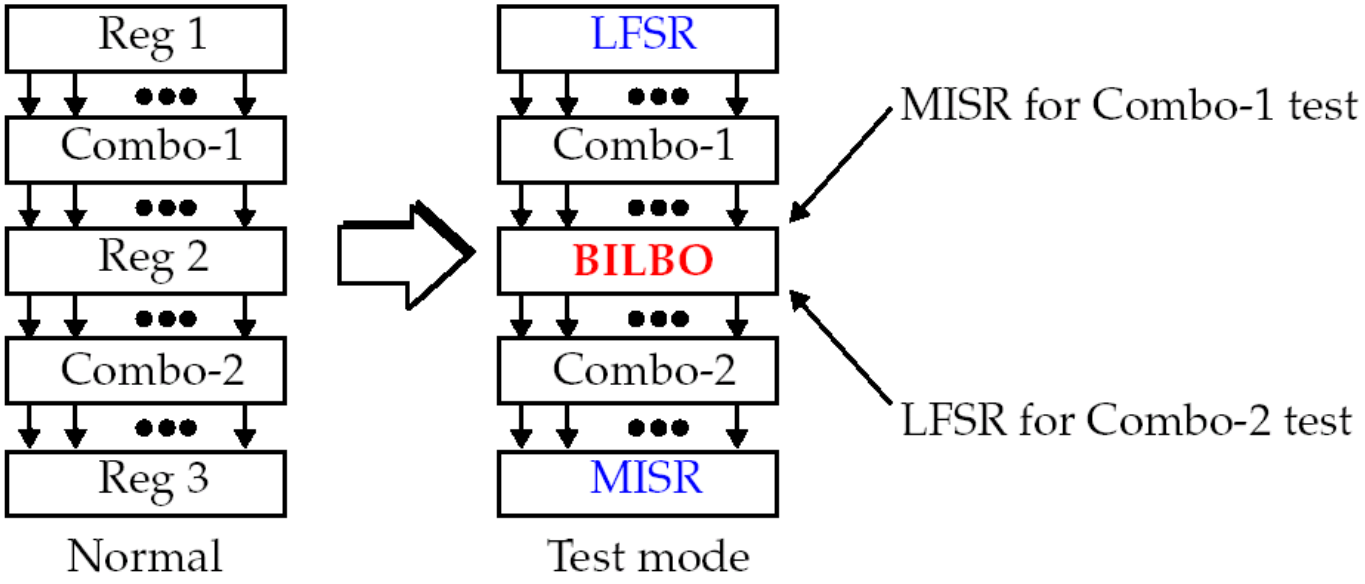


C_1 and C_2 configure as a **shift register** for scan (00), an **LFSR** (00), **MISR** (10) a **Normal** (11).

BILBO

Scan-out signatures from BILBO registers after being configured as MISRs

- Compare with expected result



Scan DFT in the Synopsys Environment

Sample Synopsys script with a concentration on DFT. (Please see the tutorial and manual for more details.)

```
read -f Verilog fsm.v
link_library = target_library = lsi_10k.db
create_clock clock81 -period 12.3

/* must expand design and click on clock81 first */
set_input_delay -clock clock81 -max -rise 2 “RW”
set_test_methodology full_scan

/* menu: attributes -> optimize directives -> design */
set_scan_style multiplexed_flip_flop

/* compile including scan */
compile map_effort low
```

Scan DFT in the Synopsys Environment...

```
/* check for testability analysis -- look at result */
insert_test
check_test

/* create test patterns to check for ATPG conflicts (additional option in */
/* create pattern menu), also checks fault coverage */
create_test_patterns -sample 5

/* full test pattern creation and scan insertion are done complete chip at */
/* the end but try them if you want */
insert_test -scan_chains 1
create_test_patterns -output fsm.vdb \
-compaction_effort low \
-check_contention_float true -backtrack_effort low \
-random_pattern_failure_limit 64 -sample 100
```

Advanced Topics in Test

Memory Test

- Even more models than discussed above

Hierarchical Testing

- To handle complexity of multicore SOCs

Test Compression

- To keep test vector set within size manageable by memory available in test equipment

Debug

- Using DFT features to help in debug of first chip

Boundary Scan

- Scan path on chip IO so chip to chip connections can be tested after assembly

Standards

- IEEE 1149 JTAG, IEEE 1500 etc.

Summary – Sub-module

Memory test

- More complex fault models including "cross-talk"
- Leads to specific patterns applied for test

Self-test

- Configuring flip-flops as LFSRs permits generation of pseudo-random test pattern sequences AND signatures of results
  - Incorrect signature → presence of a fault